AUTONOMOUS DOBOT ARM APPLICATIONS

~ System powered by Edge AI

~ Using Dobot, Hailo AI and Rasperry Pi

Presenters: Ishan Sharma, Vansh Jitendra Solanki, Sada Kakarla, Shivansh Gupta

Advisor: Prof. Chi Zhou

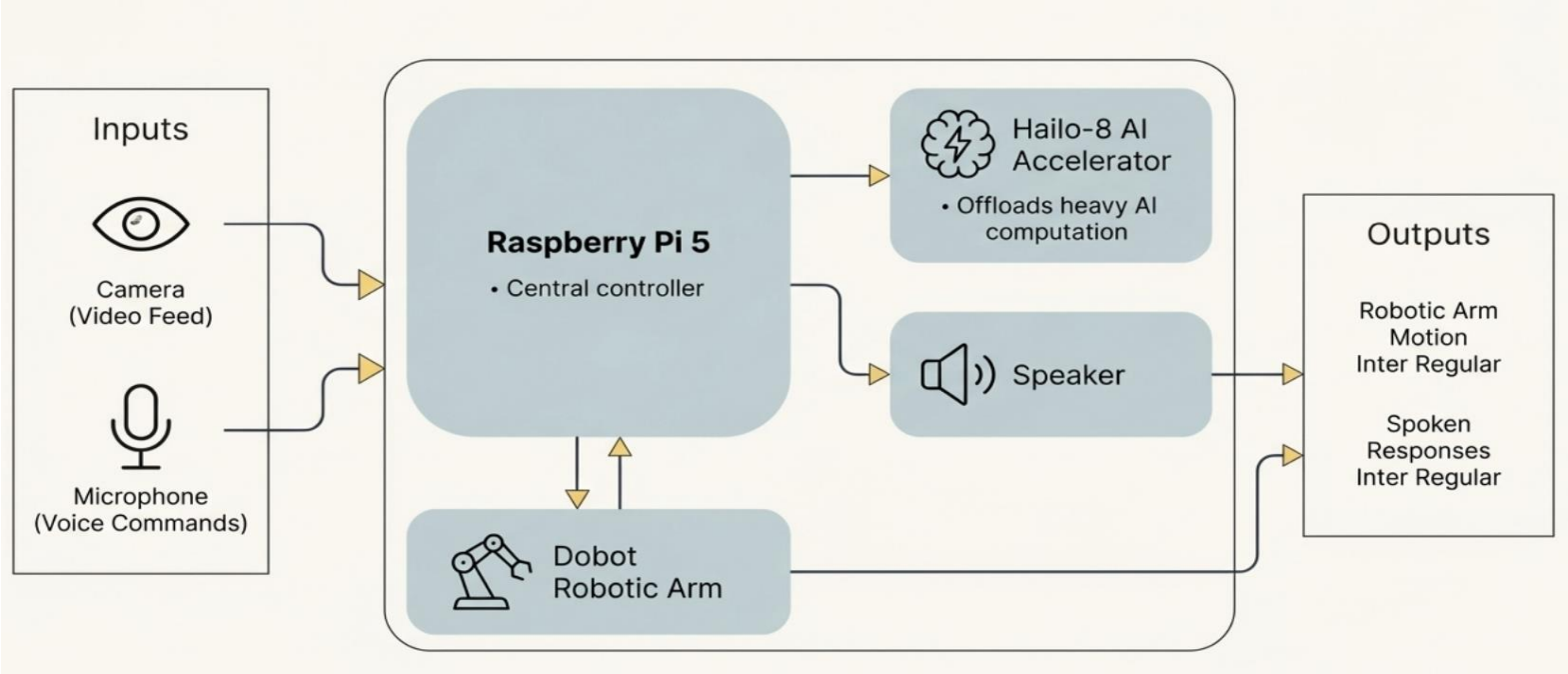
University at Buffalo The State University of New York

PROJECT OVERVIEW & OBJECTIVES

Goal 1 - Voice-Controlled Robotic Fruit Sorting

Demonstrate real-time collaboration between AI and robotics for practical automation in manufacturing

System Architecture: The Core Components



Inputs → **Camera -** Provides the live video feed for vision.
Microphone - Captures voice commands.

Outputs → **Dobot Robotic Arm:** Executes physical motion.
Speaker: Provides spoken status updates

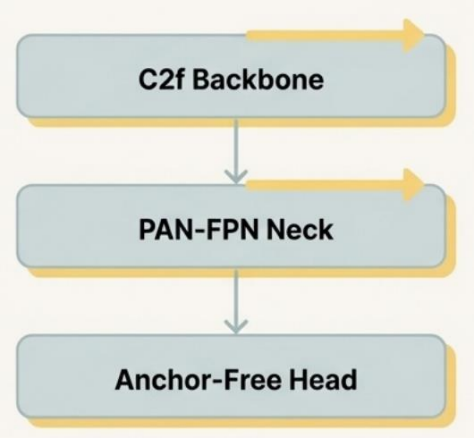
The Brain: Training a Hyper-Efficient Al Model

Primary Objective:

To build a compact, high-accuracy detection model capable of real-time performance on low-power edge devices.

Model Selection:

YOLOv8n (nano variant) selected for its superior speed-accuracy tradeoff.



Why YOLOv8n?

Superior Speed-Accuracy Tradeoff: Ideal for real-time applications.

Compact Size: A small footprint of ~6 MB with ~3 million parameters, perfect for embedded systems.

Inference Speed: Runs at ~3.9ms on the Hailo-8 accelerator, enabling real-time processing.

Modern Architecture: Features like an Anchor-Free Detection Head & PAN-FPN Neck improve detection of varied fruit sizes.

Format: The PyTorch model (.pt) is compiled into .hef format to run natively on Hailo hardware.

Data & Training: Fueling the Model with High-Quality Data

- ***Dataset Overview:*** A custom dataset was created to ensure real-world relevance, capturing images directly from the robot's camera.
- ***Categories (12 total):*** Red Apple, Green Apple, Red Grape, Black Grape, Pomegranate, Pumpkin, Corn, Eggplant, Carrot, White Radish, Strawberry, and Olive.

Annotation: Manually drawing bounding boxes for each fruit.

Augmentation: Automatically generating variations to improve model robustness.

Augmentation Transformation:
- Random flips
- Rotation & scaling
- Brightness & contrast
- Hue & saturation

Near-Human Performance: Overall Detection Accuracy

KEY PERFORMANCE INDICATORS:

- ***Inference Speed:*** ~3.9ms on the Hailo-8 accelerator, enabling real-time video processing.
- ***Model Size:*** 3,007,988 parameters (~6 MB), confirming its lightweight nature.
- ***Deployment:*** The final PyTorch model (`.pt') was compiled into the`.hef` format, optimized for native execution on the Hailo-8 hardware.

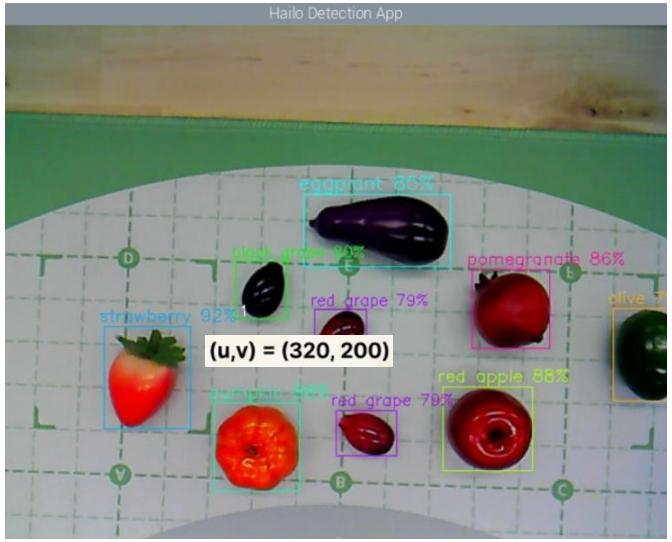
VALIDATION RESULTS:

Class	mAP50	mAP50-95
All	**0.985**	**0.843**
black grape	0.995	0.833
carrot	0.995	0.775
corn	0.995	0.755
eggplant	0.887	0.770
green apple	0.995	0.854
olive	0.995	0.864
...		

Accuracy: Achieved **98.5% mAP50** (mean Average Precision), indicating near-human detection performance

The Eyes: Translating Pixels into Reality

2D Camera View (Pixels)



Calibration

3D Robot Workspace (Millimeters)

Solution:

Camera Calibration - A process to create a precise mathematical mapping from the camera's view to the robot's workspace.

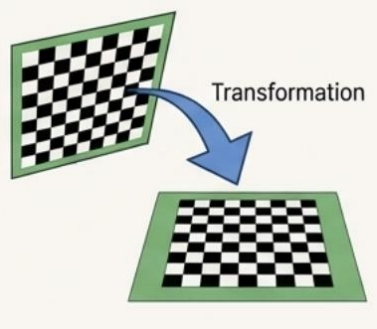
The Core Challenge:

- The AI model detects fruits in a 2D camera image (pixel coordinates u,v), but the robotic arm operates in the 3D physical world (real-world coordinates X,Y,Z).
- We need a bridge between these two coordinate systems.

Building the Bridge with Homography

What is Homography?

"Imagine the camera image as a flat sheet of paper... Homography is like a stretchable sheet transformation that bends, scales, and rotates that paper so it lines up perfectly on the real-world table."

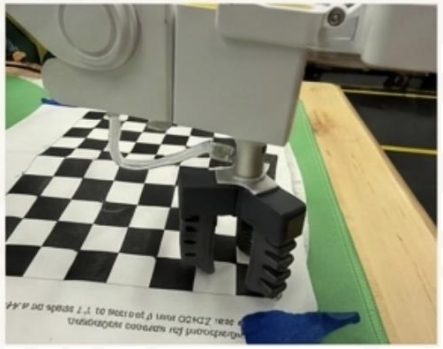


Result:

pixel_to_table_H.npy

This file enables instant conversion of any pixel location to a precise robot coordinate.

The Calibration Process



1. A user clicks a point in the camera view (capturing pixel coordinates).

2. The user moves the robot's gripper to the exact same physical spot.

3. The coordinates are paired. This is repeated with at least four points.

4. The system computes the **Homography Matrix (H),** the mathematical 'key' for the transformation.

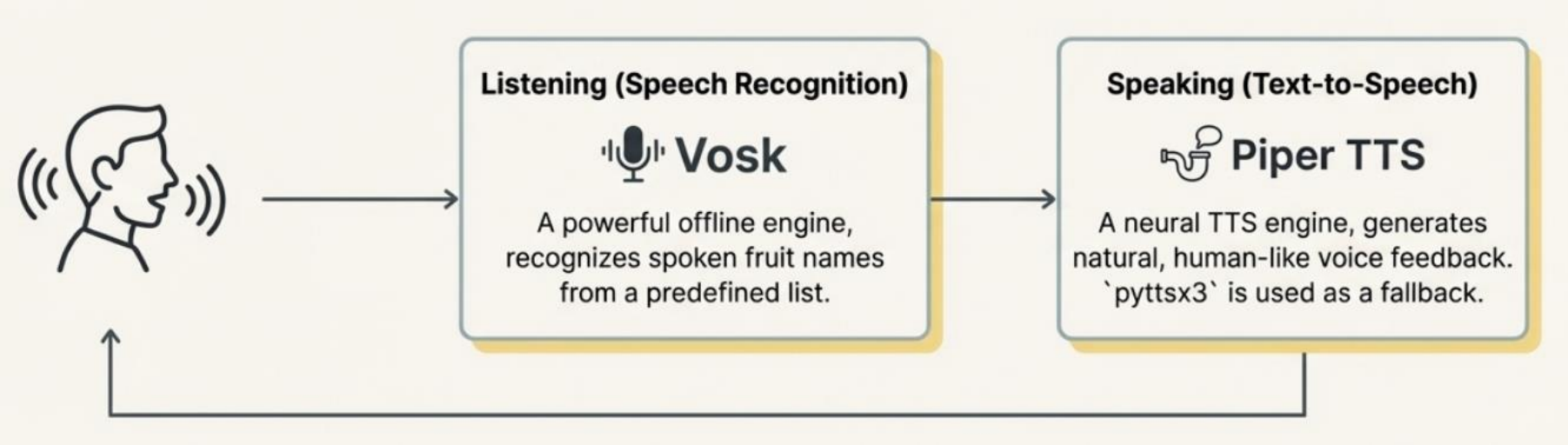
An Offline Conversational Interface

Goal:

To create a hands-free, intuitive way to control the robot without relying on keyboards, screens, or an internet connection.

Key Feature:

The entire speech pipeline runs locally on the Raspberry Pi, ensuring privacy and operation in any environment.



Synergy in Action: The Voice-Controlled Sorting Loop

Listen
User says, “Pick the strawberry.”

Understand
Vosk processes the audio and identifies “strawberry” as the target.

See
The camera feed is processed by the Hailo-8, which detects all strawberries.

Locate
The homography matrix converts each strawberry's pixel coordinates to real-world (X,Y) coordinates.

Act
The Dobot arm executes a pick-and-place sequence for each located strawberry.

Confirm
Piper TTS announces, “Okay, picking strawberry,” and later, “Done with strawberry, say another fruit.”

The Hands: Precision Pick-and-Place Motion

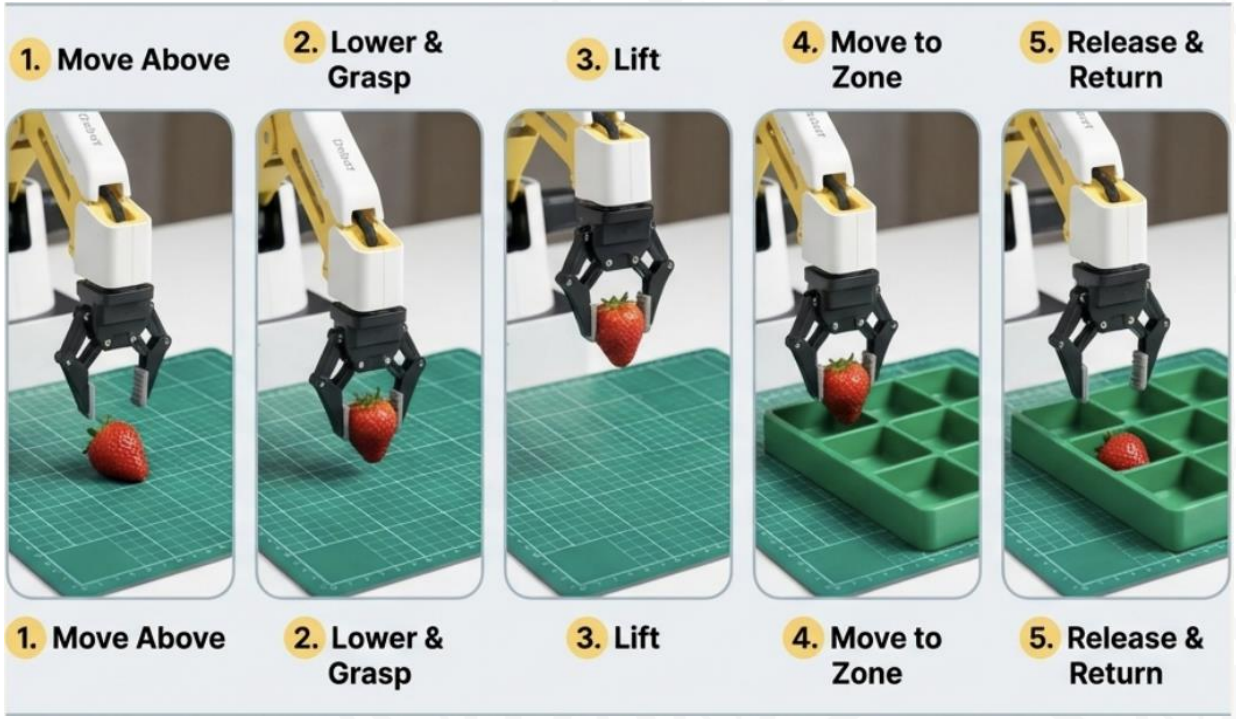
The End-Effector:

The Dobot Magician arm is equipped with a gripper for securely handling fruits of various shapes and sizes.

The Pre-Programmed Sequence:

Each pick-and-place action follows a smooth, optimized five-step motion path:

- Move above the fruit.
- Lower to the precise pick height
- Grasp the fruit.
- Lift to a safe height to avoid collisions.
- Move to the designated drop zone and release.



Scaling Up: A Continuous Sorting System with a Conveyor Belt

Objective:

To transition from a static, on-demand system to a dynamic, continuous workflow suitable for applications in packaging or agriculture.

New Components:

Motorized Conveyor Belt:
Transports items into the robot's workspace.

Proximity Sensor:
Detects when an item is in the optimal position for picking and triggers the system.

The Automated Conveyor Workflow

Transport

Conveyor moves a fruit into the camera's field of view.

Trigger & Pause

The proximity sensor detects the fruit and halts the conveyor belt.

Detect & Locate

The Hailo/YOLOv8 system instantly identifies the fruit and calculates its real-world coordinates.

Pick & Place

The Dobot arm executes its sorting routine.

Resume

Once the action is complete, the controller restarts the conveyor belt.

Technical Note: This entire process is orchestrated by a multithreaded Python controller using an `ACTION_LOCK` mutex to prevent conflicts and ensure fluid, deadlock-free operation.

Complete Intelligent System: From Edge AI to Physical Automation

This project successfully demonstrates a fully functional, AI-driven robotic sorting system capable of running entirely on embedded hardware. It combines computer vision, embedded inference, and precision mechatronics in a cohesive, self-sustaining workflow.

Brain

A hyper-efficient YOLOv8n model running at >30 FPS.

Eyes

A calibrated camera with sub-millimeter locational accuracy.

Ears & Mouth

A fully offline, interactive voice control system.

Hands

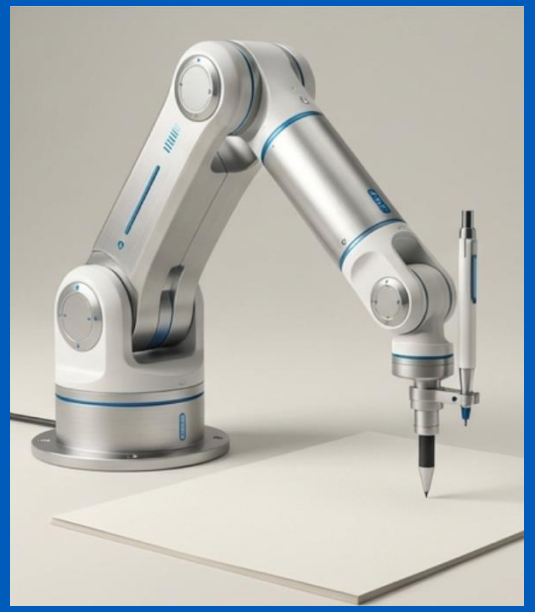
A precise robotic arm with repeatable pick-and-place actions.

"This work is a step towards smarter, voice-controlled robotic systems that can operate efficiently in industries like agriculture, manufacturing, and packaging."

PROJECT OVERVIEW & OBJECTIVES

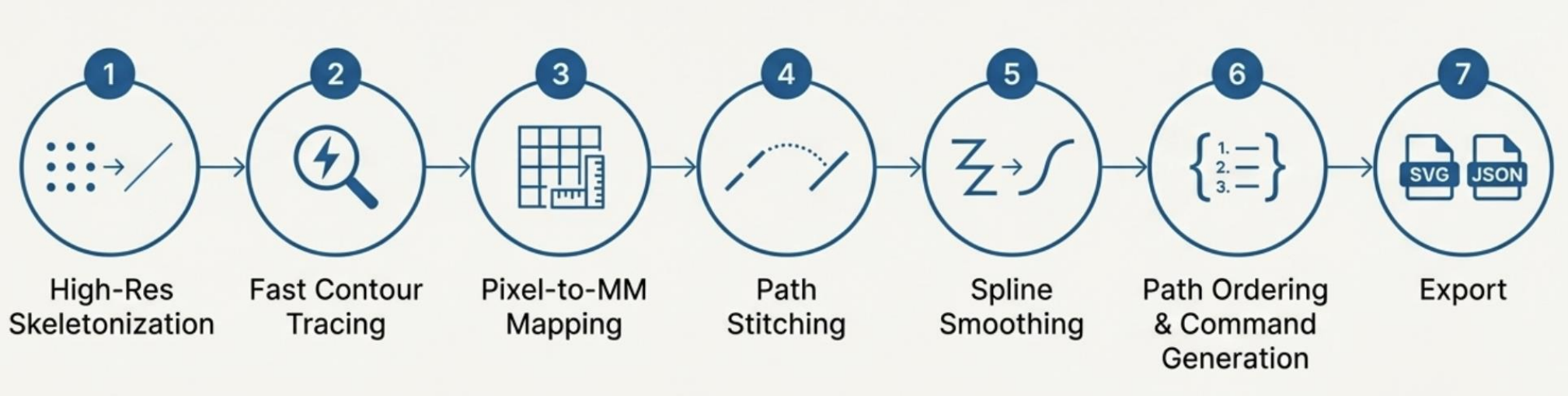
Goal 2 - High-Fidelity Image-to-Path Generation for Robotic Drawing

~ Optimizing Vectorization Pipelines for Dobot Magician



Multi-Stage Processing Pipeline

Core Objective: Convert raster images (PNG/JPEG) into smooth, efficient coordinate paths (JSON) for robotic replication.



The Processing Pipeline (Core Logic)

Step 1: Pre-processing & Upscaling:

- **4x Super-Resolution:** Images are upscaled and color is inverted to make it a binary image and preserve fine facial details (eyes, texture) before processing.
- **Adaptive Binarization:** Thresholding and morphological closing is done to solidify disjointed lines.

Step 2: Skeletonization:

- Iterative morphological erosion reduces thick lines to single-pixel centerlines (skeletons), ensuring the robot draws precise continuous strokes rather than filled shapes.

Optimization: The Shift from V5 to V6

The Bottleneck (V5 - Manual Tracing):

• Used a slow, pixel-by-pixel neighbor search algorithm (trace_skeleton). We must trace it to get ordered list of coordinates (paths)
• **Result:** High computational cost on complex images since it becomes slow while inspecting all neighbors.

The Solution (V6 - Fast Contours):

• Replaced manual tracing with **OpenCV's findContours (C++ implementation)**.
• Feed the white skeleton on a black background.
• **Performance:** ~1000x faster execution for path extraction of continuous regions with same intensity.

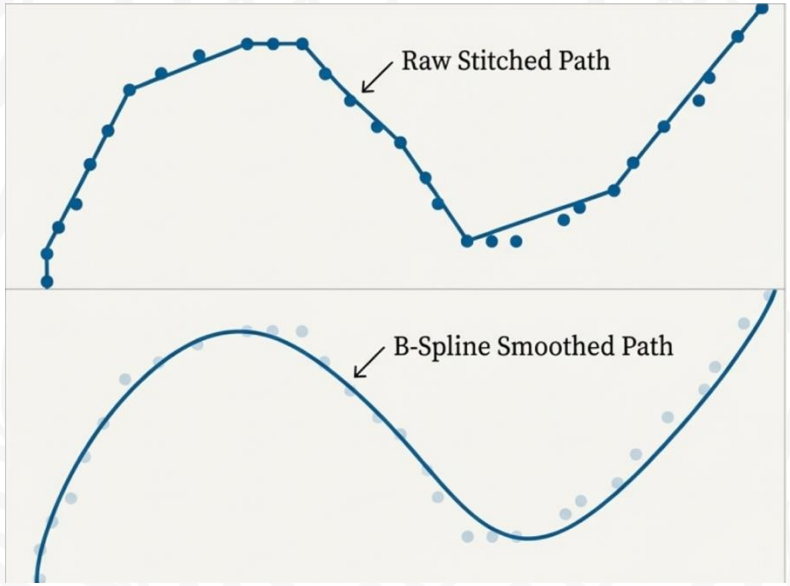
Impact: Enables processing of high-resolution inputs in seconds rather than minutes.

Path Refinement & Motion Control

Coordinate Mapping:
Transforms pixel coordinates (u, v) into physical robot workspace millimeters (X, Y) while maintaining aspect ratio.

Path Stitching:
- Connects disconnected segments that are within a specific threshold (1.0 mm).
- Reduces unnecessary "pen-up/pen-down" movements, creating smoother, continuous drawing actions.

Spline Smoothing (SciPy):
- Applies B-Spline interpolation (splprep) to raw jagged pixel paths.
- **Result:** Fluid, organic curves suitable for mechanical arm movement, removing digital "stair-stepping" artifacts.



Output Generation

The Robot's Instructions: The Final JSON Output

```
{
  "commands": [
    {
      "action": "MOVE", "x": 45.21, "y": 61.88, "pen_state": "UP"
    },
    {
      "action": "PEN_DOWN", "x": 45.21, "y": 61.88, "pen_state": "DOWN"
    },
    {
      "action": "DRAW", "x": 45.35, "y": 61.92, "pen_state": "DOWN"
    },
    {
      "action": "DRAW", "x": 45.50, "y": 61.99, "pen_state": "DOWN"
    },
    ...
  ]
}
```

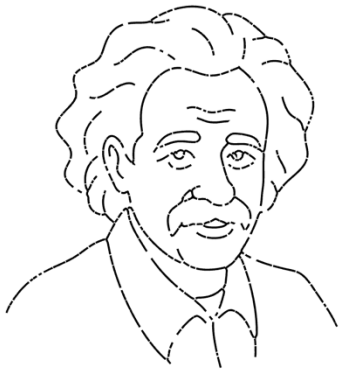
The final output is a simple, universal list of commands. Each command provides an action (`MOVE`, `PEN_DOWN`, `DRAW`) and a precise coordinate, translating the visual path into a direct set of instructions for the machine.

JSON Command Export:

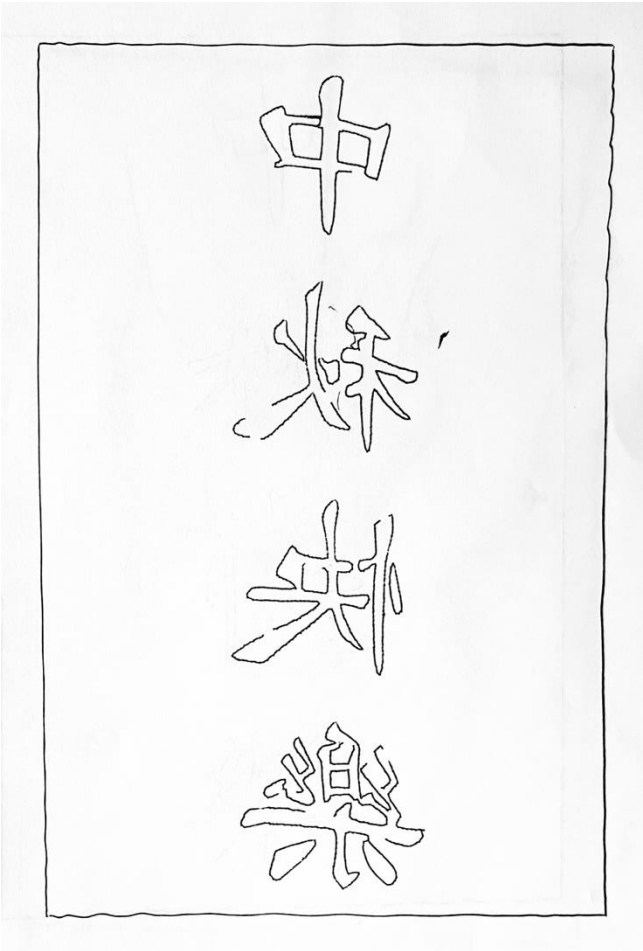
- Generates a structured list of actions: MOVE (travel), PEN_DOWN (start draw), DRAW (interpolated path).
- Optimizes the drawing order (Traveling Salesman approximation) to minimize airtime travel between lines.

Visual Validation:

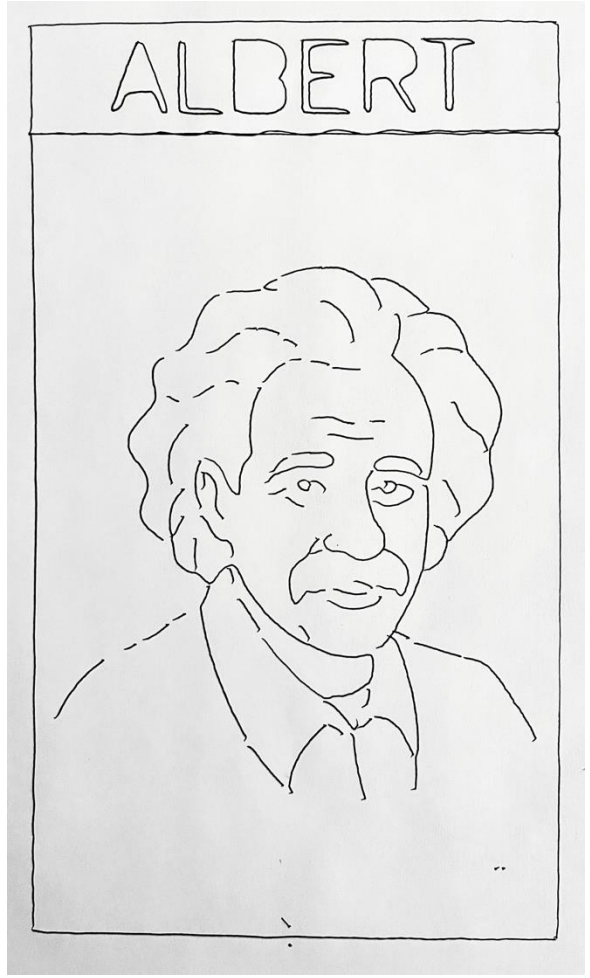
- Automatically generates an SVG preview file to verify the vector path before sending it to the physical robot.

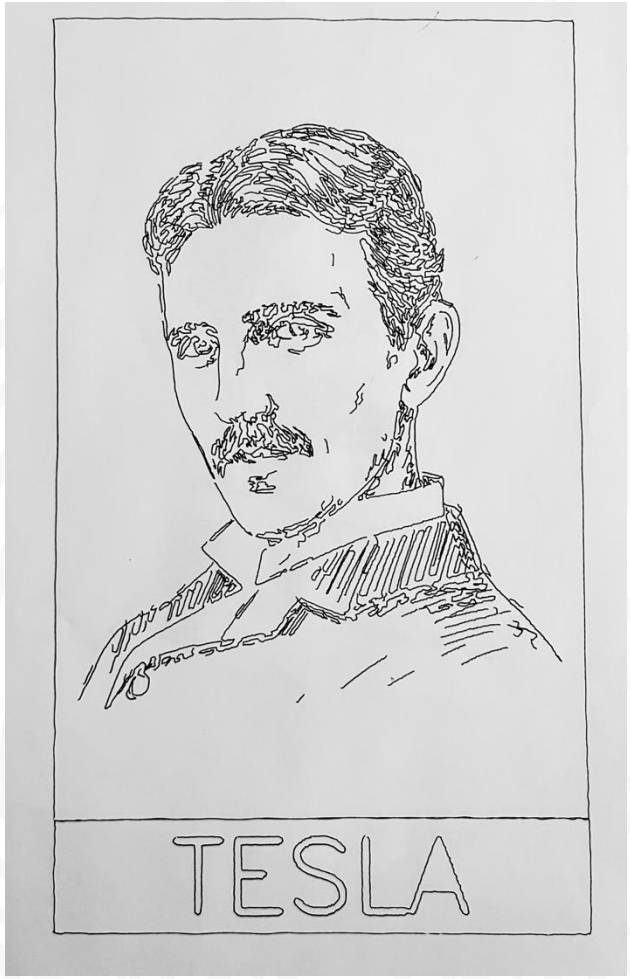


Output (Sketches drawn by Dobot):



Output (Sketches drawn by Dobot):





Future Scope Methodologies

1. Deep Learning Super-Resolution (ESRGAN/SRGAN):
• *Current:* Standard Bicubic upscaling (4x).
• *Future:* Use GANs to hallucinate realistic high-frequency details from low-res inputs before skeletonization.

2. Style Transfer for Artistic Sketch Generation
Use a pre-trained deep learning model to generate a stylized vector path.

3. Stroke Order Optimization (RL):
• *Current:* Greedy nearest-neighbor search.
• *Future:* Implement Reinforcement Learning to simulate "human-like" stroke order (e.g., drawing outlines first, then details).

4. Real-Time Feedback Loop:
Use a camera to monitor the drawing in real-time and dynamically adjust the path if the pen skips or ink runs out.

Thank You!